

Análise de Vulnerabilidades Conhecidas (CVE) em Dependências de Projetos Node.js Hospedados no GitHub

Documento de Escopo de Pesquisa — Versão 3.0

Tema da Pesquisa

Detecção e análise de vulnerabilidades conhecidas (CVE) em dependências diretas de projetos Node.js hospedados no GitHub, utilizando a base de dados do National Vulnerability Database (NVD), com avaliação de severidade, frequência e comportamento de atualização das dependências vulneráveis.

Problema

O uso de bibliotecas e dependências externas tornou-se prática comum no desenvolvimento moderno de software, especialmente em ecossistemas baseados em gerenciadores de pacotes como o Node.js/npm. Embora essas bibliotecas acelerem o desenvolvimento, podem introduzir vulnerabilidades de segurança quando apresentam falhas conhecidas ou versões desatualizadas — falhas essas catalogadas no sistema Common Vulnerabilities and Exposures (CVE).

Projetos hospedados no GitHub podem inadvertidamente utilizar dependências vulneráveis sem que os desenvolvedores percebam. Torna-se relevante investigar a presença e a severidade dessas vulnerabilidades em projetos open source, avaliando também o impacto de ferramentas automatizadas de atualização, como o Dependabot.

Escopo Delimitado

Incluído

- Projetos Node.js com arquivo package.json
- Dependências diretas declaradas no package.json (dependencies e devDependencies)
- Repositórios públicos hospedados no GitHub
- Vulnerabilidades registradas no National Vulnerability Database (NVD)
- Repositórios com e sem Dependabot habilitado (para comparação — RQ3)

Não incluído

- Outras linguagens de programação
- Dependências indiretas (transitivas)
- Repositórios privados
- Vulnerabilidades não registradas na NVD

Objetivo Geral

Investigar a incidência de vulnerabilidades conhecidas (CVE) em dependências diretas de projetos Node.js hospedados no GitHub, avaliando a frequência, severidade e distribuição dessas vulnerabilidades em um conjunto selecionado de 500 repositórios

públicos, bem como o impacto do uso do Dependabot na redução dessas vulnerabilidades.

Objetivos Específicos

1. Identificar as dependências declaradas em projetos Node.js por meio do arquivo package.json.
2. Relacionar as dependências encontradas com vulnerabilidades registradas na base NVD utilizando identificadores CVE.
3. Classificar as vulnerabilidades identificadas de acordo com seu nível de severidade (CVSS), calculando a distribuição proporcional entre os níveis baixo, médio, alto e crítico.
4. Analisar a frequência de dependências vulneráveis e desatualizadas (com correção disponível) em relação ao total de dependências dos projetos.
5. Avaliar se repositórios com Dependabot habilitado apresentam menor incidência de dependências com vulnerabilidades conhecidas em comparação a repositórios sem a ferramenta.

Perguntas de Pesquisa

RQ1

Qual é a frequência de dependências vulneráveis em projetos Node.js hospedados no GitHub? *(Analisar as dependências desatualizadas, isto é, aquelas que existe uma correção disponível como uma sub rq)*

Métricas associadas:

- M1 — Número total de repositórios analisados
- M2 — Número total de dependências identificadas (package.json)
- M3 — Número de dependências com pelo menos um CVE registrado
- M4 — Percentual de dependências vulneráveis ($M3 / M2$)
- M5 — Percentual de dependências desatualizadas com correção disponível

RQ2

Qual é o nível de severidade das vulnerabilidades encontradas nas dependências analisadas, e qual a distribuição proporcional entre os níveis de risco?

Métricas associadas:

- M6 — Distribuição de CVEs por nível de severidade CVSS (baixa, média, alta, crítica)
- M7 — Razão entre tipos de severidade (ex.: crítica+alta / total de CVEs) — Vulnerability Severity Score

RQ3

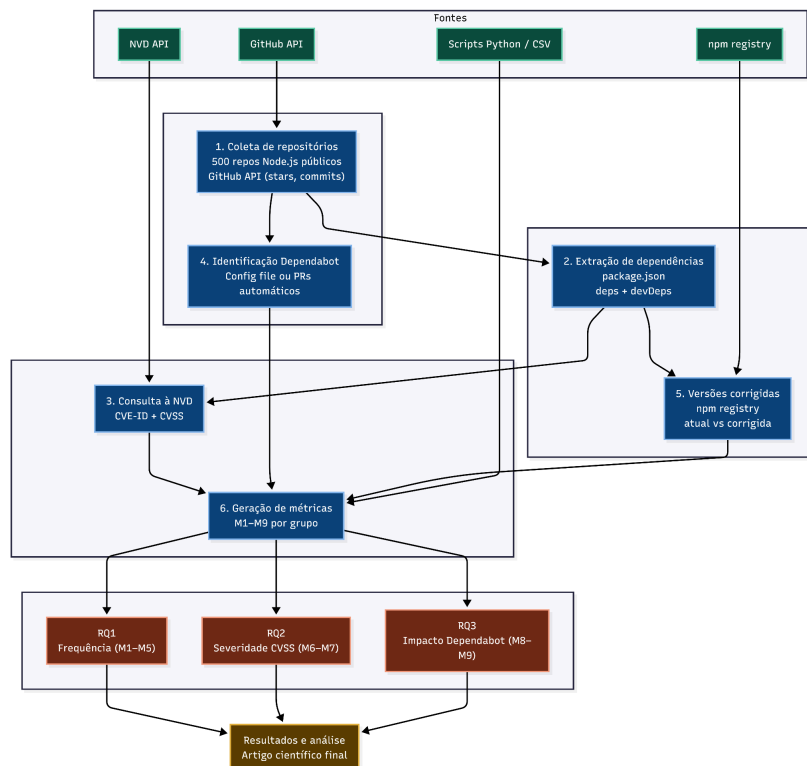
A utilização do Dependabot está associada a uma menor incidência de dependências com vulnerabilidades conhecidas nos projetos analisados?

Métricas associadas:

- M8 — Número de repositórios com Dependabot habilitado que ainda possuem dependências vulneráveis
- M9 — Comparação do percentual de dependências vulneráveis entre repositórios com e sem Dependabot
- M2, M3, M4 — aplicados separadamente para cada grupo (com/sem Dependabot)

Design do Experimento

1. **Coleta de repositórios:** Coletar 500 repositórios Node.js públicos no GitHub por meio da API do GitHub, aplicando os seguintes critérios:
 - a. Linguagem: JavaScript/TypeScript com package-lock.json na raiz;
 - b. Ordenação: repositórios ordenados de forma decrescente pelo número de stars, selecionados os 500 primeiros que satisfazem os demais critérios;ps:
 - c. Último push: menor ou igual a 12 meses antes da data de corte (15/03/2025);
 - d. Tamanho do repo: maior que 100K;
 - e. Tipo: Apenas repositórios públicos, não forks.
2. **Extração de dependências:** Extrair o arquivo package-lock.json de cada repositório e listar as dependências diretas (dependencies e devDependencies) com suas versões declaradas.
3. **Consulta à NVD:** Para cada dependência e versão identificada, consultar a API da NVD (<https://nvd.nist.gov/developers/vulnerabilities>) para verificar a existência de CVEs registrados. Registrar o CVSS score e nível de severidade de cada CVE encontrado.
4. **Verificação de atualizações:** Verificar via npm registry se existe versão mais recente (sem vulnerabilidades conhecidas) para cada dependência vulnerável encontrada.]
5. **Dependabot:** Identificar quais repositórios possuem Dependabot habilitado por dois métodos complementares:
 - a. Presença do arquivo .github/dependabot.yml via Contents API
 - b. Registro explícito no campo security_and_analysis da API de repositóriosUm repositório é classificado como "com Dependabot" se satisfizer ao menos um dos critérios acima.
6. **Geração de métricas:** Calcular todas as métricas definidas (M1–M9) e analisar os dados por grupo (com/sem Dependabot) para responder às perguntas de pesquisa.



Resultados Esperados

A pesquisa deverá permitir:

- Quantificar o nível de exposição a vulnerabilidades conhecidas em projetos Node.js open source.
- Caracterizar o perfil de severidade dessas vulnerabilidades utilizando o CVSS.
- Avaliar o comportamento de atualização de dependências vulneráveis.
- Verificar empiricamente o impacto do Dependabot na redução de vulnerabilidades em dependências.

Diagrama de Referências

NECESSÁRIO ATUALIZAR DEVIDO A TROCA DOS ARTIGOS

Referências Bibliográficas

- [1] Decan, A., Mens, T., & Constantinou, E. (2018). On the Impact of Security Vulnerabilities in the npm Package Dependency Network. MSR 2018. ACM. DOI: 10.1145/3196398.3196401
- [2] Alfadel, M., Costa, D. E., Shihab, E., & Mkhallalati, M. (2021). On the Use of Dependabot Security Pull Requests. MSR 2021. IEEE/ACM. DOI: 10.1109/MSR52588.2021.00037
- [3] Mohayeji, H., et al. (2023). Investigating the Resolution of Vulnerable Dependencies with Dependabot Security Updates. MSR 2023. IEEE/ACM. DOI: 10.1109/MSR59073.2023.00042
- [4] Liu, C., et al. (2022). Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem. ICSE 2022. ACM. DOI: 10.1145/3510003.3510142
- [5] Kumar, S. H. B. I., Sampaio, L. R., Martin, A., Brito, A., & Fetzer, C. (2024). *A Comprehensive Study on the Impact of Vulnerable Dependencies on Open-Source Software*. In **2024 IEEE 35th International

Symposium on Software Reliability Engineering (ISSRE)** (pp. 96–107). IEEE. DOI: 10.1109/ISSRE62328.2024.00020

- [6] Wu, S., et al. (2024). Vision: Identifying Affected Library Versions for Open Source Software Vulnerabilities. ASE 2024. ACM. DOI: 10.1145/3691620.3695516
- [7] Alfadel, M., Costa, D. E., Shihab, E., & Adams, B. (2023). On the Discoverability of npm Vulnerabilities in Node.js Projects. TOSEM, 32(4). DOI: 10.1145/3571848
- [8] Pashchenko, I., et al. (2018). Vulnerable Open Source Dependencies: Counting Those That Matter. ESEM 2018. ACM/IEEE. DOI: 10.1145/3239235.3268920
- [9] F. R. Cogo, G. A. Oliva and A. E. Hassan, "An Empirical Study of Dependency Downgrades in the npm Ecosystem," in IEEE Transactions on Software Engineering, vol. 47, no. 11, pp. 2457-2470, 1 Nov. 2021, doi: 10.1109/TSE.2019.2952130.
- [10] R. G. Kula, "Reducing Alert Fatigue via AI-Assisted Negotiation: A Case for Dependabot," 2025 IEEE/ACM International Workshop on Bots in Software Engineering (BotSE), Ottawa, ON, Canada, 2025, pp. 11-12, doi: 10.1109/BotSE67031.2025.00010
- [11] Kula, R. G., et al. (2018). Do Developers Update Their Library Dependencies? Empirical Software Engineering, 23(1). DOI: 10.1007/s10664-017-9521-5
- [12] Chinthanet, B., et al. (2021). Lags in the Release, Adoption, and Propagation of npm Vulnerability Fixes. Empirical Software Engineering, 26(3). DOI: 10.1007/s10664-021-09951-x
- [13] Zerouali, A., et al. (2022). On the Impact of Security Vulnerabilities in the npm and RubyGems Dependency Networks. Empirical Software Engineering, 27. DOI: 10.1007/s10664-022-10154-1
- [14] Yitagesu, S., Xing, Z., Zhang, X., Feng, Z., Bi, T., Han, L., & Li, X. (2026). *Systematic Literature Review on Software Security Vulnerability Information Extraction*. **ACM Transactions on Software Engineering and Methodology**, 35(4), Article 95. Association for Computing Machinery. DOI: 10.1145/3745026

Connected papers olhar os papers que citaram um paper específico. Pode ser útil para achar os novos artigos, avaliando as citações dos artigos antigos.